HelixQA Wiring Plan — Lava Android app on the Genymotion VM

Revision	2
Created	2026-06-08
Last modified	2026-06-08
Status	active

Table of contents

1. Scope

This document specifies how to drive a real HelixQA autonomous QA session against the **Lava Android app** (`digital.vasic.lava.client.dev`) installed on the **Genymotion VM reachable at adb `127.0.0.1:6555`**, using **Claude Code** as the LLM + Vision provider. It also enumerates the concrete gaps that must be closed before a fully autonomous, end-to-end provider-multiplexed run is possible.

The Lava-owned test banks this session consumes live at `lava-api-go/qa/banks/*.yaml` (seven banks; see §4).

> The HelixQA submodule itself (`submodules/helixqa`) is **not modified** by any step here — all configuration is supplied via env vars, CLI flags, and the Lava-owned banks directory.

2. Claude Code as the LLM + Vision provider (BridgedCLIProvider)

HelixQA already ships the bridge: `submodules/helixqa/pkg/llm/bridge_provider.go` defines `BridgedCLIProvider`, which shells out to a CLI LLM tool with `--json --print <prompt>` (plus `--model` and `--image` when set) and parses the JSON `result` / `content` / `text` field of the response.

2.1 How discovery works (zero config)

The bridge discovery is **inside the `cmdAutonomous` function** (`cmd/helixqa/main.go:404 func cmdAutonomous`), in the "Bridged CLI model discovery" block at **`cmd/helixqa/main.go:527–569`** (verified against the real `v0.2.0` binary). It iterates a fixed struct list — `{claude, qwen-coder, opencode}` (the `claude` entry is `{"claude", "claude", ""}` at **`main.go:538`**) — and for each runs `osexec.LookPath(cli.bin)` (**`main.go:542`**). If the binary is on `$PATH`, it is wrapped via `llm.NewBridgedCLIProvider(cliPath, cli.name, cli.model)` (**`main.go:546`**) and appended to the chat pool (**`main.go:553`**); when `bp.SupportsVision()` is true it is also appended to the vision pool (**`main.go:558`**).

Confirmed: `claude` IS on this host's `$PATH` at `/Users/milosvasic/.local/bin/claude` (`command -v claude`, verified this run).

Because the block lives in `cmdAutonomous`, the bridge is wired ONLY when running an autonomous QA session (`helixqa autonomous`). The `helixqa list` subcommand (`cmdList`) does NOT touch the bridge — it parses banks with no LLM, which is why listing the 7 Lava banks (§4) succeeds with zero env/credentials.

Consequences for Claude Code:

- **No env var or flag is required to enable the bridge.** Putting the `claude` binary on `$PATH` is sufficient — discovery is `osexec.LookPath("claude")` at `main.go:542`. If `claude` is not found, the loop `continues` (`main.go:544`) and the provider is simply absent; if NO API-key provider AND NO bridged CLI is found, `cmdAutonomous` exits 1 (`main.go:571–578`).
- **Vision is automatic for Claude.** `BridgedCLIProvider.SupportsVision()` returns `true` only when `cliName == "claude"` (`pkg/llm/bridge_provider.go:133–135`), so the discovered `claude` provider is added to BOTH the chat pool and the vision pool. `Vision()` (`bridge_provider.go:183`) writes each screenshot to a temp PNG and passes it via `--image <path>` (`buildArgs` at `bridge_provider.go:250–261`). This is the screenshot-analysis path the autonomous VisionEngine needs.

- **Model** is left empty ("" at `main.go:538`), so the `--model` flag is omitted (`bridge_provider.go:254–256`) and the `claude` CLI uses its configured default. To pin a model, the discovery struct's `model` field must be non-empty — see the gap in §5 (the discovery list hardcodes "").

Env/config needed for an autonomous session to use Claude

1. `claude` on `$PATH` (confirmed present here). Nothing else enables the bridge.
2. `claude --json --print "<prompt>"` MUST emit JSON the bridge can parse (`parseResponse` at `bridge_provider.go:286` reads `result` / `content` / `text`; on non-JSON it falls back to treating stdout as plain text — `bridge_provider.go:327–331`).
3. Per-invocation timeout is `defaultBridgeTimeout = 120s` (`bridge_provider.go:20`); a single bridge call may not exceed it.
4. No `ANTHROPIC_API_KEY` is required — the bridge shells out to the CLI, which carries its own auth. (An API-key provider is an *alternative*, not a prerequisite, per the `main.go:571` "no providers" guard.)

2.2 Exact invocation the bridge issues

For a chat turn the bridge runs (bridge_provider.go `buildArgs`):

```
claude --json --print "<concatenated prompt>"
```

For a vision turn (screenshot analysis):

```
claude --json --print "<prompt>" --image /tmp/helixqa-vision-
XXXX.png
```

Per-invocation timeout defaults to 120 s (`defaultBridgeTimeout`).

2.3 Pre-flight checklist

```
command -v claude                  # MUST resolve — discovery is
        LookPath("claude")
claude --json --print "ping"       # MUST return JSON with a
        result/content/text field
```

If `claude --json --print` does not emit a JSON object, the bridge falls back to treating stdout as plain text (`parseResponse`), which still works for chat but is lower-fidelity; prefer a `claude` build that honours `--json`.

3. Pointing an autonomous session at the Lava app on the Genymotion VM

3.1 Inputs

Input	Value	Supplied via
App package	`digital.vasic.lava.client.dev`	env `HELIX_ANDROID_PACKAGE`
Device serial	`127.0.0.1:6555` (Genymotion)	env `HELIX_ANDROID_DEVICE`
Platforms	`android`	flag `--platforms android`
Project root	repo root (feature map from docs)	flag `--project`
Env file	`.env` (gitignored, real secrets)	flag `--env`
Output dir	`qa-results/`	flag `--output`
LLM + Vision	Claude Code on `$PATH`	auto-discovery (§2)

The env-var names are read directly in `cmd/helixqa/main.go`: `AndroidPackage: os.Getenv("HELIX_ANDROID_PACKAGE")` (~line 662) and `AndroidDevice: os.Getenv("HELIX_ANDROID_DEVICE")` (~line 660). The autonomous flags (`--project`, `--platforms`, `--env`, `--timeout`, `--coverage-target`, `--output`, `--report`, `--curiosity`, `--curiosity-timeout`) are defined in the `autonomous` flagset (~line 404).

3.2 Connect adb to the Genymotion VM

Genymotion exposes adb over TCP. Connect the host's adb server to it first so the serial `127.0.0.1:6555` appears in `adb devices`:

```
adb connect 127.0.0.1:6555
adb devices                          # MUST list 127.0.0.1:6555 as
        'device' (not 'offline')
adb -s 127.0.0.1:6555 shell pm list packages | grep
        digital.vasic.lava.client.dev
```

If the package is not installed, install the debug APK first:

```
adb -s 127.0.0.1:6555 install -r app/build/outputs/apk/debug/app-
        debug.apk
```

3.3 The `.env` (real secrets — gitignored per §6.H / §11.4.10)

Create a `.env` at the location passed to `--env`. It MUST NOT be committed. Real tracker credentials are referenced BY NAME in the banks and read at runtime here:

```
# Device / app targeting
HELIX_ANDROID_DEVICE=127.0.0.1:6555
HELIX_ANDROID_PACKAGE=digital.vasic.lava.client.dev

# Tracker credentials consumed by the per-provider banks (NEVER
hardcoded in YAML)
RUTRACKER_USERNAME=...
RUTRACKER_PASSWORD=...
RUTOR_USERNAME=...
RUTOR_PASSWORD=...
KINOZAL_USERNAME=...
KINOZAL_PASSWORD=...
NNMCLUB_USERNAME=...
NNMCLUB_PASSWORD=...
# archiveorg + gutenberg are auth_type NONE — no credentials.

# LLM + Vision: none required — Claude Code is auto-discovered on
$PATH (§2).
```

3.4 Launch command

```
cd /Volumes/T7/Projects/Lava
adb connect 127.0.0.1:6555

submodules/helixqa/bin/helixqa autonomous \
  --project . \
  --platforms android \
  --env ./.env \
  --timeout 2h \
  --coverage-target 0.9 \
  --output qa-results/ \
  --report markdown,html,json
```

Outputs land under `qa-results/` (report + tickets + videos), suitable as the `docs/qa/<run-id>/` end-user evidence required by §11.4.83.

3.5 The session's 4 phases (what actually happens)

Per the HelixQA README, the autonomous session runs: (1) **Setup** — model selection + feature map from `--project` docs + spawn agents + init VisionEngine; (2) **Doc-Driven Verification** — workers verify documented features against the running app, capturing screenshots/video; (3) **Curiosity-Driven Exploration**; (4) **Report & Cleanup**. The

NavigationEngine's Android `ActionExecutor` drives the device over adb (taps/swipes/screencap); Claude (vision) interprets each screenshot to decide the next action and to verify the `expected` of each bank step.

4. Test banks consumed by the session

Seven Lava-owned banks, all `platforms: [android]`. **Verified against the real `helixqa v0.2.0` binary** (built `go build -o /tmp/helixqa ./cmd/helixqa`, exit 0): all 7 banks load with ZERO YAML fixes required — the banks were already schema-correct for the real `pkg/testbank` loader (the earlier PyYAML-only check is now superseded by real-binary proof). 32 test cases total.

4.1 VERBATIM real-binary `helixqa list` output

```
$ /tmp/helixqa list --banks /Volumes/T7/Projects/Lava/lava-api-go/
qa/banks
Test cases: 32

ID           NAME                                              CATEGORY
PRIORITY   PLATFORMS
------------------------------------------------------------------------
LAVA-ARCHIVEORG-001 Onboard and select the Internet Archi...
functional   critical   android
LAVA-ARCHIVEORG-002 Continue anonymously without hanging ...
functional   critical   android
LAVA-ARCHIVEORG-003 Search Internet Archive for a realist...
functional   high       android
LAVA-ARCHIVEORG-004 Open an item and obtain a working HTT...
functional   critical   android
LAVA-GUTENBERG-001 Onboard and select the Project Gutenb...
functional   critical   android
LAVA-GUTENBERG-002 Continue anonymously without hanging ...
functional   critical   android
LAVA-GUTENBERG-003 Search Project Gutenberg for a realis...
functional   high       android
LAVA-GUTENBERG-004 Open an item and obtain a working dow...
functional   critical   android
LAVA-KINOZAL-001 Onboard and select the Kinozal provider
functional   critical   android
LAVA-KINOZAL-002 Authenticate to Kinozal with form-log...
functional   critical   android
LAVA-KINOZAL-003 Search Kinozal for a realistic query ...
functional   high       android
LAVA-KINOZAL-004 Open a result and obtain a working do...
functional   critical   android
LAVA-NAV-001 Cold-start survival (no crash on firs... functional
critical   android
LAVA-NAV-002 Welcome brand mark renders in colour,... ux
high       android
```

```
LAVA-NAV-003 Full onboarding happy path reaches th... functional
critical   android
LAVA-NAV-004 Back-press on the first onboarding sc... functional
critical   android
LAVA-NAV-005 Configure multiple providers during o... functional
high       android
LAVA-NAV-006 Open Trackers from Settings without a... functional
high       android
LAVA-NAV-007 Theme switch and rotation preserve state ux
medium     android
LAVA-NAV-008 Primary in-app navigation reaches sea... functional
high       android
LAVA-NNMCLUB-001 Onboard and select the NNM-Club provider
functional   critical   android
LAVA-NNMCLUB-002 Authenticate to NNM-Club with form-lo...
functional   critical   android
LAVA-NNMCLUB-003 Search NNM-Club for a realistic query...
functional   high       android
LAVA-NNMCLUB-004 Open a result and obtain a working do...
functional   critical   android
LAVA-RUTOR-001 Onboard and select the RuTor provider
functional   critical   android
LAVA-RUTOR-002 Authenticate to RuTor with form-login...
functional   critical   android
LAVA-RUTOR-003 Search RuTor for a realistic query an...
functional   high       android
LAVA-RUTOR-004 Open a result and obtain a working do...
functional   critical   android
LAVA-RUTRACKER-001 Onboard and select the RuTracker prov...
functional   critical   android
LAVA-RUTRACKER-002 Authenticate to RuTracker with captch...
functional   critical   android
LAVA-RUTRACKER-003 Search RuTracker for a realistic quer...
functional   high       android
LAVA-RUTRACKER-004 Open a result and obtain a working do...
functional   critical   android
```

The `--platform android` variant (`list --banks ... --platform android`) emits the same 32 cases (all Lava cases declare `platforms: [android]`), in load order rather than sorted. Both invocations exit 0.

The 7 banks map to the table below:

Bank	Provider	auth_type	Download deliverable verified
`lava-rutracker-journey.yaml`	rutracker	CAPTCHA_LOGIN	torrent file / magnet
`lava-rutor-journey.yaml`	rutor	FORM_LOGIN	torrent file / magnet
	kinozal	FORM_LOGIN	

Bank	Provider	auth_type	Download deliverable verified
`lava-kinozal-journey.yaml`			torrent file / magnet
`lava-nnmclub-journey.yaml`	nnmclub	FORM_LOGIN	torrent file / magnet
`lava-archiveorg-journey.yaml`	archiveorg	NONE	HTTP file link (no torrent/ magnet)
`lava-gutenberg-journey.yaml`	gutenberg	NONE	torrent file or HTTP file
`lava-onboarding-navigation.yaml`	(cross-cutting)	—	cold-start / gating / nav

Auth types and download capabilities are taken from the real descriptors under `core/tracker/<provider>/.../<Provider>Descriptor.kt`.

To reproduce the listing through the real binary:

```
cd /Volumes/T7/Projects/Lava/submodules/helixqa
go build -o /tmp/helixqa ./cmd/helixqa
/tmp/helixqa list --banks /Volumes/T7/Projects/Lava/lava-api-go/
        qa/banks --platform android
```

5. What is still MISSING (concrete task list)

These are the concrete gaps between "banks exist + bridge exists" and "a fully autonomous provider-multiplexed run executes against the Genymotion VM". None of them are solved by this document; each is an actionable task.

1. **RESOLVED — HelixQA binary now builds and lists the banks.** The earlier blocker (unresolved go.mod conflict markers + uninitialized own-org deps) is closed: 5 own-org dependency submodules were added, and `cd submodules/helixqa && go build -o /tmp/helixqa ./cmd/helixqa` now exits 0 and produces `helixqa v0.2.0`. `helixqa list` against `lava-api-go/qa/banks` loads all 7 banks / 32 cases (verbatim output in §4.1). No bank-YAML fix was needed — the banks already match the real `pkg/testbank` loader schema. `helixqa autonomous` is NOT exercised here (it needs a booted device/emulator, which is not available this run — see §3.2/§5.4/§5.5 for the remaining wiring).

2. **Navigator adapter for Lava-specific screens (Q6).** The autonomous `pkg/navigator` Android `ActionExecutor` drives generic adb taps/swipes and relies on Vision to locate controls. Lava's Compose screens expose specific semantics (onboarding step controls, provider rows, the captcha field, the torrent vs magnet download affordances). **Task:** define a Lava navigator adapter / semantics map (resource-ids or Compose `testTag`s for: provider list rows, login username/password/captcha fields, search input, result row, topic download + magnet controls) so the navigator can act deterministically instead of vision-guessing, with vision as the fallback. This is the open "Q6 navigator adapter" item.

3. **Session config for the bridge + device is not templated.** There is no committed `.env.example` enumerating `HELIX_ANDROID_DEVICE`, `HELIX_ANDROID_PACKAGE`, and the per-tracker credential VAR NAMES for a Lava autonomous run. **Task:** add a `lava-api-go/qa/.env.example` (placeholders only, no secrets — §6.R/§6.H) documenting every variable §3.3 lists, plus a `--model` pin decision for the bridge (the discovery block currently hardcodes model `""`; pinning a Claude model requires either a small upstream change to the discovery list or an env-driven override — itself a HelixQA-side task).

4. **Per-provider multiplexing / orchestration is not wired.** A single `helixqa autonomous` invocation points at one app + one feature map; it does not iterate the six provider banks as distinct credentialed journeys, nor reset app state (`pm clear`) between providers. **Task:** add a Lava-side thin runner (e.g. `lava-api-go/qa/run-provider-matrix.sh`) that, per provider: `adb connect 127.0.0.1:6555` → `pm clear digital.vasic.lava.client.dev` → export that provider's credential vars → invoke the session (or `helixqa run`) scoped to that provider's bank → collect `qa-results/<provider>/`. This is the multiplexing layer that turns seven banks into one matrix run.

5. **adb-over-TCP reachability gate.** The session assumes `127.0.0.1:6555` is in `adb devices` as `device`. **Task:** add a pre-flight step to the runner (#4) that runs `adb connect` + asserts the serial is online and the package is installed, failing loud (not silently testing nothing) per the anti-bluff posture.

6. **Download-payload verification is described, not yet asserted programmatically.** The banks specify "non-empty bencoded torrent" / "magnet:?xt=urn:btih:" / "HTTP 200 non-empty payload" as the `expected`. **Task:** ensure the navigator (or a post-step hook) actually captures the produced torrent bytes / magnet URI / HTTP file and validates the shape — so a green step proves a real working download, not merely that a button was tappable (§6.G / §11.4.69 sink-side evidence).

6. Constraints honoured

- **§6.R / §6.H / §11.4.10** — no credentials in any tracked file. The banks and this doc reference credential VARIABLE NAMES only; real values live in the gitignored `.env` loaded at runtime.
- **submodules/helixqa unmodified** — all configuration is external (env, flags, Lava-owned banks). The build blocker in §5.1 is reported honestly, not worked around by editing the submodule.
- **CONST-046** — banks describe structure + intent prose; they do not hardcode the exact user-facing literal strings the app must display as assertion-equality values.

Sources verified

Real-binary evidence captured 2026-06-08 with `helixqa v0.2.0` (`go build -o /tmp/helixqa ./cmd/helixqa`, exit 0):

- **`helixqa list --banks lava-api-go/qa/banks [--platform android]`** — exit 0, 32 cases across the 7 Lava banks (verbatim in §4.1). Supersedes the prior PyYAML-only validation.
- `submodules/helixqa/pkg/llm/bridge_provider.go` — bridge invocation (`buildArgs` :250, `--model` omit :254, `--image` :257), vision gating (`SupportsVision` :133), JSON/plain-text parse (`parseResponse` :286, fallback :327), `defaultBridgeTimeout = 120s` :20.
- `submodules/helixqa/cmd/helixqa/main.go` — bridge discovery inside `cmdAutonomous` (:404): block :527–:569, `{"claude","claude",""}` :538, `osexec.LookPath(cli.bin)` :542, `NewBridgedCLIProvider(...)` :546, chat-pool append :553, vision-pool append :558, no-provider exit guard :571–:578. Autonomous flags + `HELIX_ANDROID_*` env-var reads as noted in §3.1.
- **`command -v claude`** → `/Users/milosvasic/.local/bin/claude` (claude IS on `$PATH`, so the bridge would self-discover in an autonomous session).
- `submodules/helixqa/README.md` — bank schema, autonomous 4-phase model, `list`/`autonomous` CLI.
- `core/tracker/<provider>/.../{<Provider>Descriptor}.kt` — per-provider id, displayName, authType, capabilities.